What we built

Slabora | Team Astrova | ModelX 2.0 Implementation

Slabora is a working browser application for comparing concrete pour start times using hourly forecast weather. It combines a deterministic evaporation estimate, explicit temperature and wind scenarios, rain exposure during an early review period, and rules that refuse a lower-risk label when their inputs do not justify one. It is an educational planning prototype, not structural approval or a crack-prediction model.

Delivered user workflow

Select Colombo, Kandy, Galle, Jaffna or pilot-area coordinates; fetch live weather; inspect the next 48 hourly intervals; set a workday and 1-6 hour review period; compare ranked candidates; inspect reasons and scenario ranges; export an auditable CSV. The Overview, Pour planner and How it works views share the same model. The chart supports 24/48-hour horizons, evaporation/rain tabs and a numeric hourly table.

Architecture and code boundaries

Layer / file	Responsibility
Static host / dist/index.html	Serves the application shell. There is no application backend, account system or database.
dist/weather.js	Requests Open-Meteo; validates timezone, units, arrays and hourly timestamps; supplies explicitly labelled synthetic sample data.
dist/core.js	Pure functions for scenarios, evaporation, completeness/freshness checks, state assignment, ranking, local time and CSV.
dist/app.js + styles.css	Browser state, fetch timeout, controls, explanations, hand-drawn SVG charts, responsive navy/sage interface and accessibility behavior.
tests/ + scripts/ + evidence/	29 automated tests, dated provider snapshots, a repeatable frozen-data replay and a new-response capture utility.

Stack chosen and why

The submitted application uses JavaScript ES modules, HTML, CSS, SVG, Fetch and Intl. Node.js built-in tests verify the calculation without third-party packages. Python is only an optional local HTTP server and a standard-library evidence-capture script. There is no npm install or compilation step. This supports simple static hosting and separates the numeric core from presentation.

This changes the proposed Python/Streamlit, requests, Pandas and Plotly stack. The benefit is a portable static application with no server process. The tradeoff is custom interface/chart code and direct dependence on browser access to the weather service. No persistent forecast cache or scheduled refresh service was added.

Calculation and decision rules

Implemented equation and input scenarios

E = max(0, 5 × 10^-6 × [(Tc + 18)^2.5 - (RH/100) × (Ta + 18)^2.5] × (V + 4))

E is kg/m²/h; air temperature Ta and concrete temperature Tc are °C; RH is percent; near-surface wind V is km/h. The formula is the metric simplification selected in the confirmed proposal [1]. Invalid or non-finite inputs are rejected; negative evaporation is clamped to zero.

Unknown Tc creates three cases: Ta, Ta+5°C and Ta+10°C. Each is combined with 0.65 × 10 m mean wind and a separate 0.65 × 10 m gust stress input: six combinations per hour. A user-supplied concrete reading replaces the three temperature cases, leaving two wind cases. The offsets and wind conversion are uncalibrated assumptions; a reading is not a prediction of future surface temperature.

Time alignment and complete periods

For an hourly interval [t,t+1h), temperature, humidity and mean wind are sampled at t. Precipitation, probability and maximum gust are taken from timestamp t+1h because those provider fields describe the preceding hour [2]. For a candidate [start,end), every required hourly interval must exist. The period uses maximum evaporation over all hours and scenarios, maximum hourly rain probability, and summed accumulation. Probabilities are never added.

State	Default implemented rule
Cannot assess	Any invalid/missing critical value, incomplete period, missing retrieval timestamp, retrieval more than 3 hours old, or more than 60 seconds in the future.
Elevated concern	Conservative E ≥1.0 kg/m²/h, maximum rain probability ≥60%, or period accumulation ≥1 mm.
Lower modeled risk	All tested E <0.5 kg/m²/h, maximum rain probability <30%, and exactly zero accumulation; all data checks passed.
Review conditions	All other complete, fresh cases. Rain thresholds are configurable pilot screens, not damage probabilities.

Candidate ranking

The default workday is 06:00-18:00 and duration is three hours. Both start and end must fit the selected workday. Candidates are sorted by lower-risk, review, elevated and unavailable states; ties use conservative evaporation, rain probability, accumulation, then start time. The chart horizon and candidate date filter are independent controls.

The proposal described an upper band as above 1.0. The delivered rule conservatively includes exactly 1.0 in elevated concern; boundary tests document this choice. Scenario ranges are sensitivity envelopes, not confidence intervals or calibrated uncertainty distributions.

Data used, temporality and gaps

Source and final data dictionary

The application calls the public Open-Meteo Forecast API over HTTPS for selected latitude/longitude. It requests four calendar days to cover 48 future hourly starts plus the endpoint needed for interval alignment, using Asia/Colombo, Unix timestamps and explicit km/h wind units. It uses forecast grid data, not a thermometer, rain gauge or slab sensor [2].

Provider field / unit	Role and treatment
temperature_2m / °C	Air temperature at 2 m; equation input at interval start.
relative_humidity_2m / %	Humidity at 2 m; divided by 100 inside the equation.
wind_speed_10m / km/h	Mean wind; multiplied by 0.65 for the baseline scenario.
wind_gusts_10m / km/h	Preceding-hour gust maximum; separate stress scenario.
precipitation_probability / %	Preceding-hour forecast probability; use maximum over the review period, never a slab-damage probability.
precipitation / mm	Preceding-hour accumulation; sum over the review period.
time / Unix seconds	Converted to milliseconds internally and displayed in Asia/Colombo, UTC+05:30.

Freshness, storage and failure behavior

Live weather loads on entry and on explicit refresh or location change. The browser timeout is 18 seconds; a request identifier prevents an older response overwriting a newer request. A timer checks age and local-hour transitions, but does not periodically refetch. The three-hour cutoff is an application policy, not a promised provider model-update schedule.

Forecasts and current location remain in memory. Only planning settings persist in localStorage. Missing values remain missing; there is no critical-input interpolation or persistent forecast fallback. A failed request clears the old forecast and displays an error. Sample mode must be selected explicitly and is visibly labelled synthetic. Custom coordinates are bounded to latitude 5-10 and longitude 79-82; this is a rectangle, not a geographic land mask.

Evidence captured on 10 November 2026

An actual HTTP 200 response was saved at 16:42:12 UTC (22:12:12 Sri Lanka). It contains 96 hourly rows; the production parser and model produced 48 valid future intervals and 20 default-workday candidates. Saved request metadata and a frozen-clock replay make this result reproducible. The response is now a dated forecast snapshot, not current advice or observed weather. A separate 4 November fixture is also retained.

Gaps: no measured concrete-surface temperature, local shielding/wind, solar exposure, mix/bleeding properties, slab geometry, actual curing/protection, observed site weather or labelled crack/washout outcomes. Retrieval time is recorded; underlying forecast-model issuance and full provider lineage are not pinned. Forecast revisions and grid mismatch remain sources of error. Open-Meteo attribution is retained; public free access is subject to its non-commercial terms and limits [3].

Evaluation against the proposal

Evidence available

On 10 November 2026, node --test tests/*.test.mjs passed all 29 tests with zero failures. All three application modules also passed Node syntax checks. Coverage includes three independently calculated values of the exact equation (absolute tolerance 1e-12), boundary values, six scenarios, measured-input mode, late-period hazards, precipitation/gust alignment, six missing-field cases, incomplete/stale/future data, Sri Lanka midnight, workday/ranking constraints, API units/schema and CSV provenance.

The fresh Colombo replay returned 4 review and 16 elevated candidates, with no lower-risk window. Its top candidate was 11 November 06:00-09:00 local time: conservative E 0.6612 kg/m²/h, maximum hourly rain probability 6%, and accumulation 0 mm. This is an integration example from one forecast, not evidence of forecast or concrete-outcome accuracy.

Proposal commitment	Delivered result / remaining gap
Hourly forecasts + transparent calculation	Implemented with live provider access, selected equation, scenarios, interval reasoning, candidate ranking and CSV.
Python / Streamlit stack	Changed to browser JavaScript, HTML/CSS/SVG. Pure calculation functions retain the proposed separation of concerns.
Caching and stale-data handling	Freshness checks and explicit error states delivered. Persistent caching and SQLite were not implemented.
Calculation verification	29 software tests pass. Decimal gold values verify the selected equation; a separate reproduction of the original published nomograph/example is not documented.
Baseline and forecast evaluation	No completed comparison with a generic-weather baseline, archived-forecast study, observation-based MAE or rainfall calibration score.
3-5 external-user walkthroughs	Not completed. Development browser walkthroughs are not a substitute for user research.
Professional/field validation	Not completed. No concrete outcomes, expert approval, savings estimate, predictive superiority or novelty claim is established.

Manual development checks and limits

Earlier browser checks covered real retrieval, settings and re-ranking, reset, navigation, chart tabs, explicit sample labels, a CSV download, the hourly table and a narrow mobile layout. These are development checks, not a formal cross-browser, accessibility or external-user study. No coverage percentage or performance benchmark is claimed.

The defensible result is a reproducible decision-support workflow with software-level safeguards. It does not establish that its top candidate is optimal, that the weather forecast is accurate, or that cracking will be prevented. The missing empirical evaluations are the main shortfall against the original research plan.

Reproduce, inspect and extend

Run the submitted code

Requirements: a current browser; Python 3 for the optional local server; Node.js 20+ for verification. No application package installation, API key or build step is needed. From the repository root run:

python3 -m http.server 3000 --directory dist
Open http://localhost:3000

On Windows, use py instead of python3 if that is the installed launcher. Serve over HTTP; opening the HTML directly can block ES-module loading. For static hosting, publish dist/ or use the provided repository-root entry page for GitHub Pages.

npm test
npm run check
node scripts/replay-evidence.mjs

The first two commands verify tests and JavaScript syntax. The replay evaluates the saved real response at its recorded capture time and regenerates evidence/replay-results.json and a CSV. It is intentionally separate from the live app and never presents that old response as fresh. python3 scripts/capture-weather.py fetches a new response and records its actual retrieval timestamp.

Next work justified by the evidence

First, obtain construction-informed review of the equation use, wind/temperature assumptions, screening thresholds and wording. Then compare the scenario-aware ranking against an explicit weather-only baseline on saved forecast snapshots. With suitable observations, assess lead-time weather error and rain calibration. Run the planned 3-5 formative user tasks, recording comprehension and task completion. Field-outcome work would require an appropriate supervised study and data collection; no training dataset presently exists.

AI assistance and accountability

ChatGPT/Codex assisted with proposal interpretation, design, implementation, tests, source review and submission documentation. The team remains responsible for reviewing the code and report, understanding the assumptions, verifying the submission, crediting data and disclosing assistance under the competition rules. No machine-learning model or LLM is used to calculate the application outputs.

References and evidence locations

[1] Team Astrova. Slabora_ModelX2.0_Proposal - Team Astrova(1).pdf, team-confirmed Phase 1 proposal supplied for this comparison. Its personal contact details are not republished in this repository.

[2] Open-Meteo. Weather Forecast API documentation. https://open-meteo.com/en/docs (checked 10 November 2026).

[3] Open-Meteo. Terms and data attribution. https://open-meteo.com/en/terms. Weather data by Open-Meteo, CC BY 4.0; free public API subject to applicable terms and limits.

[4] Formula background cited in the proposal: P. J. Uno, Plastic Shrinkage Cracking and Evaporation Formulas, ACI Materials Journal 95(4), 365-375 (1998); ACI Committee 305, Guide to Hot Weather Concreting, ACI 305R-20. This report evaluates the selected software equation, not independent engineering validation of those publications.

Repository evidence: tests/core.test.mjs; evidence/test-results-2026-11-10.txt; evidence/retrieval-metadata.json; evidence/openmeteo-colombo-2026-11-10.json; evidence/replay-results.json. Submission guide and timed narration: SUBMIT_NOW.md and DEMO_SCRIPT.md.